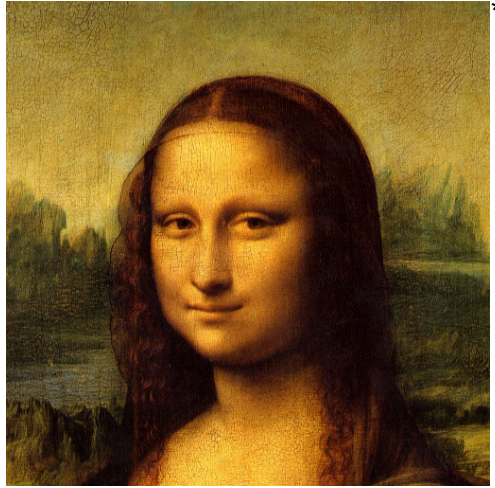




Project Proposal

Sean Larsen - seala24@student.sdu.dk

Kristóf Ákos Koltai - akkol24@student.sdu.dk

Denis Nedzhibov - dened24@student.sdu.dk

Igor Zimmermann - igzim24@student.sdu.dk

Örs Tomaj Jeney - orjen24@student.sdu.dk

Dávid Borka - dabor24@student.sdu.dk

* The Mona Lisa is considered as Da Vinci's magnum opus.
This project is our magnum opus.

Contents

1. Background and Motivation	3
2. Aim	4
3. Use cases	5
3.1. Booking process	5
3.2. Users interacting during the consultation process	5
3.3. Review and send doctor's note to patient	6
4. Initial Requirements	7
4.1. Functional Requirements	7
4.2. Non-Functional Requirements	7
5. Methods	9
5.1. Task tracking	9
5.2. Documents and Presentations	9
5.3. Repository	9
6. Architecture	10
6.1. Application Layer Diagram	10
6.1.1. Frontend Layer	10
6.1.2. Backend Layer:	11
6.1.3. Infrastructure Services:	12
6.1.4. Data Layer:	12
6.2. Component-Based System Diagram	12
6.3. Proposed Tech Stack	13
7. Risks	14
7.1. Security	14
7.2. Componentized approach	14
Appendix	15

1. Background and Motivation

Outpatient departments face outdated manual processes that drastically slow down operations and put a burden on both patients and medical staff. Doctors and nurses spend too much time on paper schedules, taking notes during consultations, and using separate systems for writing prescriptions, which causes long wait times, a poor patient experience (due to disrupted doctor focus), errors, and burnout from extra administrative work. These issues also create serious privacy risks for patient data, which demand strict precautions and careful handling.

Our project is relevant because it offers a solution for a clear gap: the need for simple AI tools that run entirely locally using open-source models, keeping all sensitive health information secure within the clinic's network. By automating the full process from patient booking and real-time transcription to AI-generated clinical summaries, "safety-net" suggestions, and instant digital prescriptions, it helps healthcare facilities modernize quickly and affordably. This reduces mistakes, speeds up patient flow, and relieves medical staff of the burden of endless paperwork, allowing them to focus on delivering quality care and improving patient satisfaction.

2. Aim

In many outpatient departments, doctors must manually manage appointments, transcribe consultations, and write prescriptions, which disrupts the flow of patients and increases the workload of hospital staff. Our project addresses this by providing a privacy-preserving, locally hosted OPD management system that automates the workflow from consultation to digital prescription using open-source speech-to-text and large language models, with a “safety net” of suggestive alerts.

The technical aim of our project is to tackle the challenge of component-based architecture by implementing multiple independent services, such as appointment management, transcription, clinical summarization, medical suggestions, and prescription generation. These services are isolated, each has its own runtime environment with well-defined interfaces that connect them and enable easy replacement and seamless upgrades of individual components.

The system follows a design that prioritizes privacy preservation because dealing with sensitive health data requires strict precautions. To achieve this privacy-first design, we will only use locally hosted open-source models, so that patient data never leaves the hospital’s network.

This modular design makes it easy to replace components while preserving privacy and supporting clinical workflow.

3. Use cases

3.1. Booking process

- Primary actor (user)
 - Patient
- Preconditions
 - Patient is registered in the system
 - Patient has valid login credentials
- Goal
 - Book a consultation with a doctor
- Flow, see Appendix Figure C, Appendix Figure D
 - User navigates to the client website
 - User logs in
 - User selects a preferred doctor
 - The system displays available time slots
 - The user books an available appointment
 - The user is redirected to a dashboard where they can:
 - See their booking(s)
 - Remove their booking(s)
 - The system sends a confirmation email

3.2. Users interacting during the consultation process

- Primary actor (user)
 - Doctor
- Secondary actor
 - Patient
- Preconditions
 - Appointment exists in the system
 - Doctor has valid credentials
- Goal
 - Doctor and patient discuss the issues
 - Doctor identifies symptoms, makes a diagnosis, and if necessary prescribes medicine and refers the patient to another department
 - The system internally processes the audio and transcribes the consultation
- Flow, see Appendix Figure E
 - User navigates to the doctor website
 - User logs in
 - The system displays a dashboard with today's bookings related to the user
 - The user selects the appropriate appointment
 - The user starts the consultation
 - The system starts the audio recording
 - The system waits until the user ends the consultation

- Audio recording is stopped and sent for processing by the speech-to-text component

3.3. Review and send doctor's note to patient

- Primary actor (user)
 - Doctor
- Preconditions
 - Consultation recording completed and successfully transcribed
- Goal
 - Review AI-generated documentation, validate clinical suggestions, and send the final doctor's note
- Flow, see Appendix Figure E
 - The system generates a summary based on the transcription
 - The system displays the consultation dashboard:
 - Transcription summary
 - The user can either edit or accept the summary
 - When the summary is accepted, the system generates suggestions based on the accepted summary
 - Suggestions include:
 - Medicine
 - Referral to other departments
 - The system displays two views:
 - Suggestions on one half
 - Doctor's note/prescription on the other half
 - The doctor's note includes:
 - Symptoms
 - Diagnosis
 - Prescriptions
 - The user can edit the doctor's note/prescription

4. Initial Requirements

The following requirements define the core functionality of the project. The prioritization follows the MoSCoW method.

4.1. Functional Requirements

- **MUST have**

- The system shall allow patients to create, view, and cancel appointment bookings.
- The system shall provide secure login for patients and doctors.
- The system shall record and transcribe consultations locally using an open-source speech-to-text model.
- The system shall generate a structured consultation summary and prescription using a locally hosted open-source LLM.
- The system shall implement a “Suggestive Mode” that analyzes the transcript summary and suggests potential missing clinical elements.
- Doctors shall be able to review, edit, approve, or reject AI-generated summaries and prescriptions.
- The system shall generate a PDF prescription after a doctor’s approval.
- The system shall send the approved prescription to the patient via a secure email service.
- The system shall store structured data (users, bookings) in a relational database.
- The system shall store transcripts and AI-generated content in a non-relational database.
- The system shall support user registration through secure email invitations.

- **SHOULD have**

- The system should log AI prompts and outputs for evaluation purposes.
- The system should allow configurable instructions for the Suggestive Mode.
- The system should allow users to search for a doctor and see available doctors.
- The system should allow doctors to start and end consultations.

- **COULD have**

- The system could allow doctors to upload diagnostic images as part of a consultation.
- The system could provide simple statistics about the number of consultations and bookings.
- The system could support switching between different local LLM models.

- **WON'T have**

- The system will not use any cloud-based AI services.
- The system will not include a mobile application.

4.2. Non-Functional Requirements

- **Performance**

- The system shall transcribe consultation audio with minimal delay under normal hardware conditions.
- The system shall generate and deliver a PDF prescription within 5 seconds after approval.
- Booking-related requests shall respond within 3 seconds under normal usage.
- **Portability**
 - The system shall run identically on Windows, Linux, and macOS.
 - All components shall be containerized for consistent deployment.
- **Scalability**
 - The system shall support at least 5 concurrent consultations without service failure.
 - Each infrastructure service shall be independently replaceable without modifying other components.
- **Maintainability**
 - The project shall follow a CBSE-aligned architecture, where services and components are loosely coupled and communicate through clearly defined API interfaces.
 - Each service or component shall be replaceable or upgradeable with minimal impact on the rest of the system.
 - The modular structure shall simplify debugging by isolating faults to specific services/components.
 - The defined interfaces and separation of concerns shall improve long-term maintainability and make components interchangeable.
- **Security**
 - All patient and consultation data shall remain within the local network.
 - User passwords shall be securely hashed before storage.
 - Role-based access control shall prevent unauthorized access to consultation data.
 - Separate systems for user booking and doctor consultation shall prevent unauthorized access to consultation data.
- **Reliability**
 - The system shall recover automatically from container failure.
 - No transcript or approved prescription shall be lost due to service interruption.
- **Testing Coverage**
 - Core backend logic shall include unit and integration tests.
 - At least 80% of core application logic shall be covered by automated tests.

5. Methods

Before starting out on our project, we outlined a set of rules and methods to ensure that progression through the project will be smooth and continuous. The methodology and tools used also ensure that every contribution made to the project is peer-reviewed.

5.1. Task tracking

In order to ensure that none of the tasks get lost, we make every task (development, diagramming, documenting) an issue on Jira. We complete these tasks during one-week-long Sprints, from one Monday to the next. On Mondays, we hold meetings where we both reflect on the Sprint ending that day and plan the one coming up. On Thursdays, we also hold a stand-up meeting where everyone gives an update on the issue(s).

To ensure that tasks are distributed fairly, we assign tasks not based on quantity, but based on their actual difficulty, which we estimate using Story Point Poker and the Fibonacci numbering system.

5.2. Documents and Presentations

To create our documentation and presentations, we chose Typst. It allows us to version control and handle our documents as if they were code. Templates have also been made to create a uniform design and look for all of our reports and presentations. In order to keep track of our images and diagrams, we also decided to store them in the same repository as our documents and code. We also create a meeting log document during each meeting to keep everyone accountable and for anyone to catch up if they were absent.

5.3. Repository

To make sure code contributions are safe, each issue has its own branch, and pull requests must be opened for merging. Those pull requests must then be reviewed by at least two non-contributing peers. In order to have time for review and changes before the end of the sprint, pull requests must be opened at least two days before the end of the sprint. To help the work of the reviewers and to guarantee a smooth workflow, a Pull Request template has also been made, see Appendix Figure F. We merge all pull requests during our Monday meetings, so we can resolve possible merge conflicts together. We also created a pipeline that sends a message to our Discord about a new pull request, and by replying to that message, the PR owner tags the requested reviewers. This is done because not everyone checks their emails, which is GitHub's default notification channel.

6. Architecture

This semester's project will be structured using a layered architecture, implementing a component-based system (CBS) framework focusing on modularity, replaceability, and reusability. Components communicate via an API Gateway implementing a service-oriented pattern.

6.1. Application Layer Diagram

This section analyzes the Application diagram, see Figure 1. It breaks down the individual layers of this layered architecture.

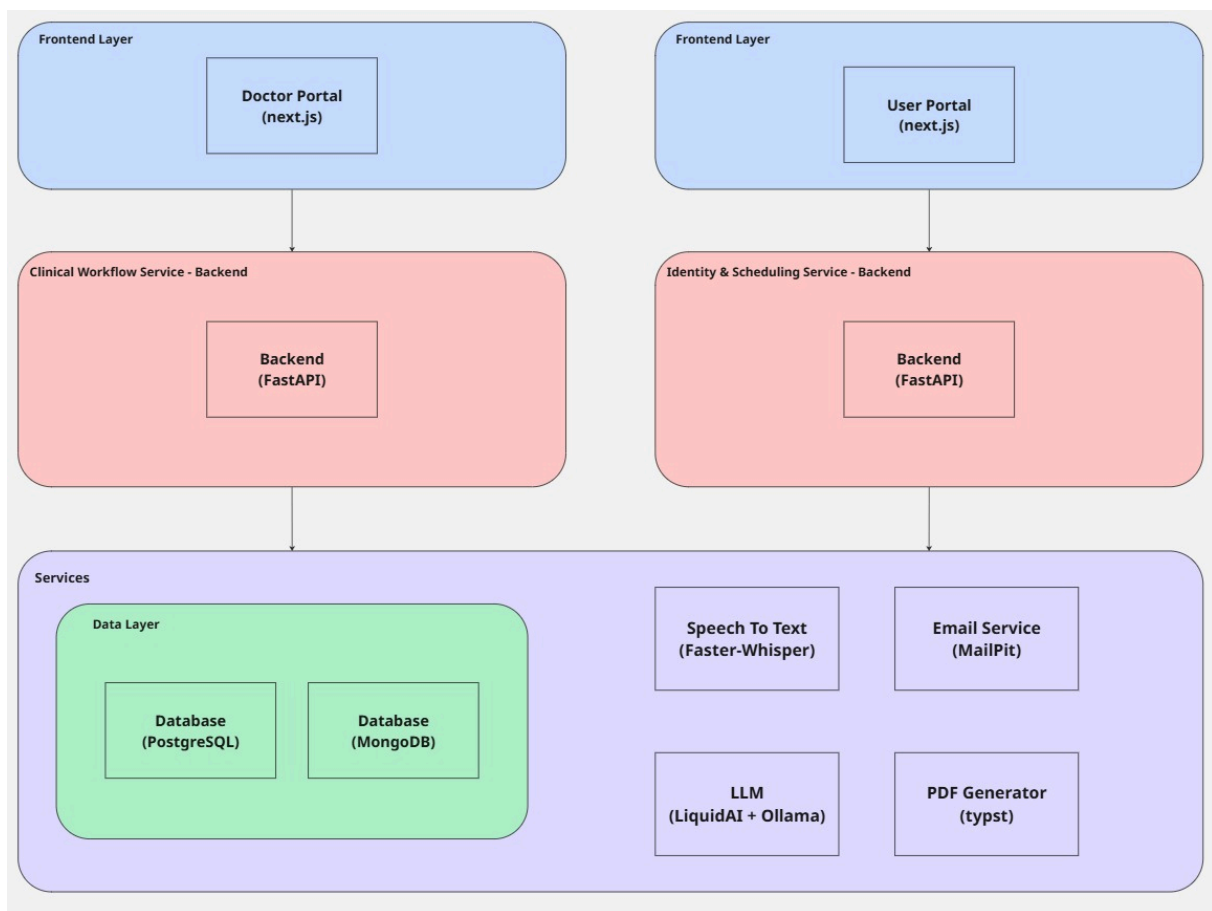


Figure 1: Application Layer Diagram

6.1.1. Frontend Layer

The main responsibility of this layer is to act as an intermediary between the user and the backend services by triggering API calls to the orchestration logic.

- **User Portal (Next.js/TypeScript):** Provides a user interface for booking management and authentication for clients. Allows users to search for doctors, see availability, and create/remove bookings.
- **Doctor Portal (Next.js/TypeScript):** Provides a doctor interface for selecting bookings and starting consultations. Allows the doctor to record consultations, edit

transcriptions, read LLM-generated suggestions, and send prescriptions to patients.

6.1.2. Backend Layer:

For this project, the backend layer is split into different systems. One is for the user booking system, the other for the doctor consultation system. The reason for this design choice is to ensure privacy first. By separating the system into two, any calls made to the booking system only handle booking data, protecting client medical data.

- **Clinical Workflow Service:** This backend service focuses on the doctor workflow, including the following business logic:
 - Authentication: authenticates logins and requests.
 - Transcription: sends consultation audio files to the speech-to-text service.
 - Summary Editor: allows the doctor to edit the LLM-generated summary.
 - Suggestions Editor: allows the doctor to review LLM-generated suggestions produced with the draft prescription, and manually adjust the prescription accordingly.
 - Send Prescription: exports the prescription as a PDF and emails it to the patient.
- **Booking Service:** This backend service focuses on booking management, logins, and registrations, including the following business logic:
 - Authentication: authenticates logins, requests, and registrations.
 - Availability: checks for doctor availability.
 - Booking: manages patient bookings by creating or removing them.
- **Clinical Workflow Service:** This backend service focuses on the doctor workflow, including the following business logic:
 - Authentication: authenticates logins and requests.
 - Transcription: sends consultation audio files to the speech-to-text service.
 - Summary Editor: allows the doctor to edit the LLM-generated summary.
 - Suggestions Editor: allows the doctor to review LLM-generated suggestions produced with the draft prescription, and manually adjust the prescription accordingly.
 - Send Prescription: exports the prescription as a PDF and emails it to the patient.
- **Booking Service:** This backend service focuses on booking management, logins, and registrations, including the following business logic:
 - Authentication: authenticates logins, requests, and registrations.
 - Availability: checks for doctor availability.
 - Booking: manages patient bookings by creating or removing them.

6.1.3. Infrastructure Services:

The infrastructure services exposed through stable HTTP interfaces provide specialized functions. These services are isolated Docker containers exposed via HTTP interfaces for modularity and follow the CBS framework.

The infrastructure services exposed through stable HTTP interfaces provide specialized functions. These services are isolated Docker containers exposed via HTTP interfaces for modularity and follow the CBS framework.

- **Speech-to-Text (Faster-Whisper):** Transcribes audio files
- **LLM Runtime (Ollama + model):** Ollama acts as an intermediary between HTTP requests and the LLM. We have yet to decide on a specific model.
- **PDF Generation (Typst):** Produces exportable PDF files.
- **Secure Email (Mailpit):** Sends generated outputs to corresponding clients.

6.1.4. Data Layer:

The main responsibility of this layer is data persistence.

- **Relational Database (PostgreSQL):** Stores structured data such as users, doctors, and booking information.
- **Non-relational Database (MongoDB):** Stores unstructured data such as transcripts, accepted AI suggestions, AI summaries, and generated doctor's notes.

6.2. Component-Based System Diagram

The Component-Based System (CBS), see Figure 2, illustrates how the system is structured into modular components with exposed interfaces. It can be broken down into three main layers: Frontend, Backend, and Services/Databases. This design allows individual components to be independently developed and interchanged at runtime.

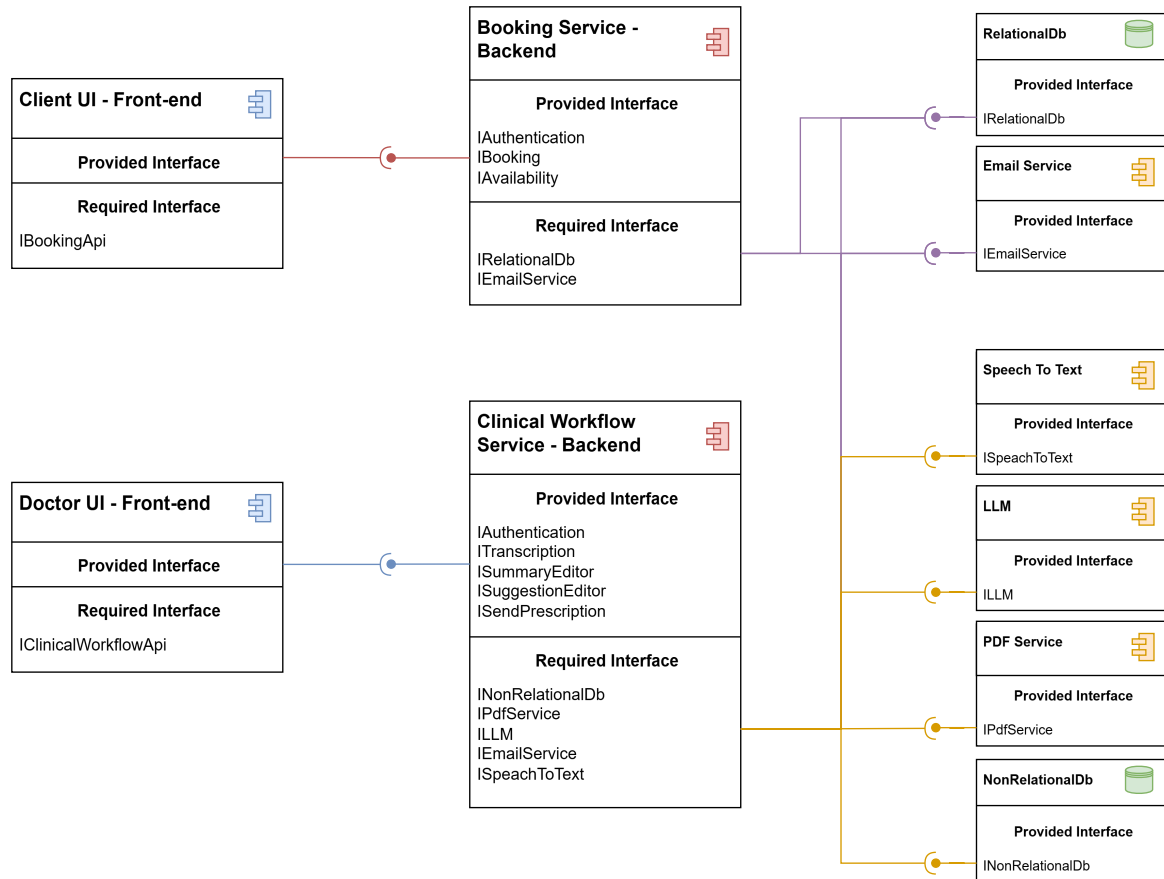


Figure 2: Component-Based System Diagram

6.3. Proposed Tech Stack

Layer	Technology	Purpose
Frontend	Next.js / TypeScript	User/Doctor interface
Backend	FastAPI	Orchestration & business logic
Speech-to-Text	Faster-Whisper	Audio transcription
LLM Runtime	Ollama + LLM models	Local AI
PDF Generator	Typst	PDF generation
Email	Mailpit	Email sending
Data	PostgreSQL + MongoDB	Un/structured storage
Deployment	Docker Compose	Containerized services & networking
CI (OPTIONAL)	GitHub Actions	Automated test & build checks

Table 1: Application tech stack

7. Risks

In this section we will identify potential risk and security factors. As this project will be used in professional medical environments, making sure that sensitive personal data is handled correctly is our top priority.

7.1. Security

To ensure this, we will be implementing two separate systems. Our booking system for patients to manage bookings will be cloud-hosted, and a doctor system will be hosted locally. By separating the system, we ensure confidential information stays local.

Additionally, we will store booking system data in a separate relational database from the sensitive client information stored in a non-relational database. All sensitive information moving within the network will be encrypted to ensure data security.

Our system will be locally hosted, including the LLM, which means no data will be sent to external servers, further reducing the risk of data breaches.

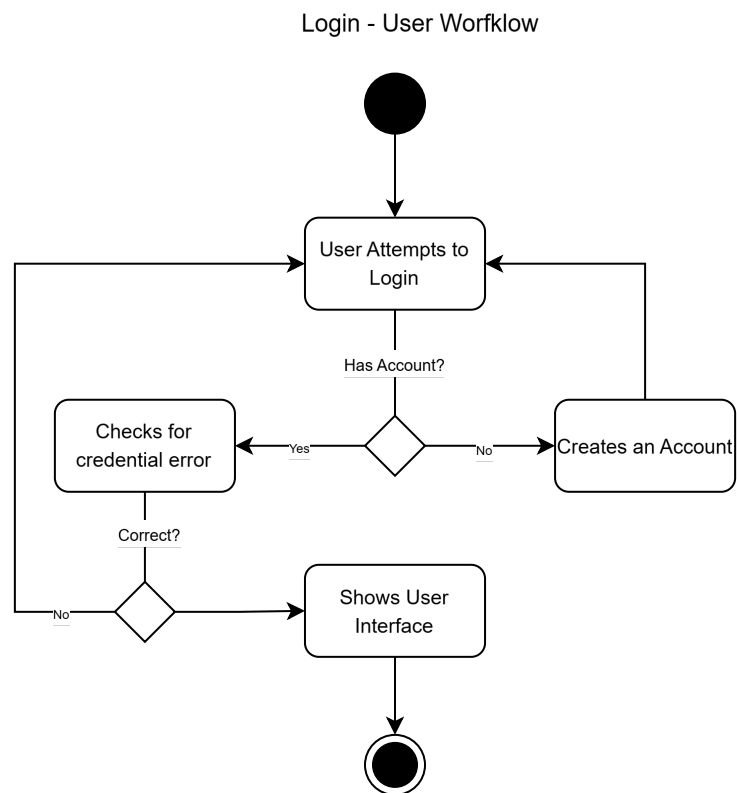
Our backend will implement token authentication, so unauthorized access attempts will be rejected before they can reach the services. Authentication and authorization will be handled by the authentication manager, which will be responsible for ensuring that users can only access appropriate data based on their permissions.

7.2. Componentized approach

The componentized architecture addresses the risk of tight coupling. By breaking down the whole system into smaller components and separating responsibilities, we can ensure that if one fails, others can still function. This approach also allows us to easily find issues and replace components without affecting the whole system.

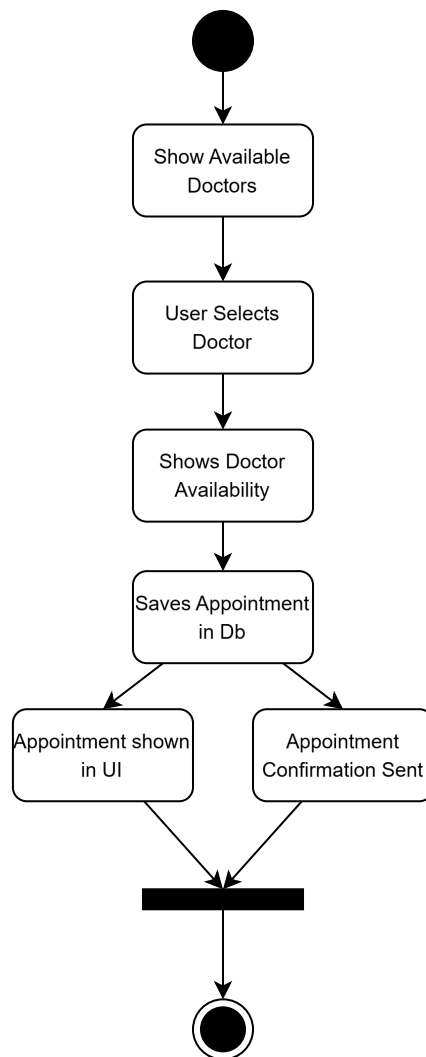
To achieve this, we will use Docker containerization to isolate each component. This allows us to scale components as needed, and run health checks and automatic recovery. This means we can periodically check the availability of components and automatically restart them if they fail to respond, reducing downtime and improving reliability.

Appendix

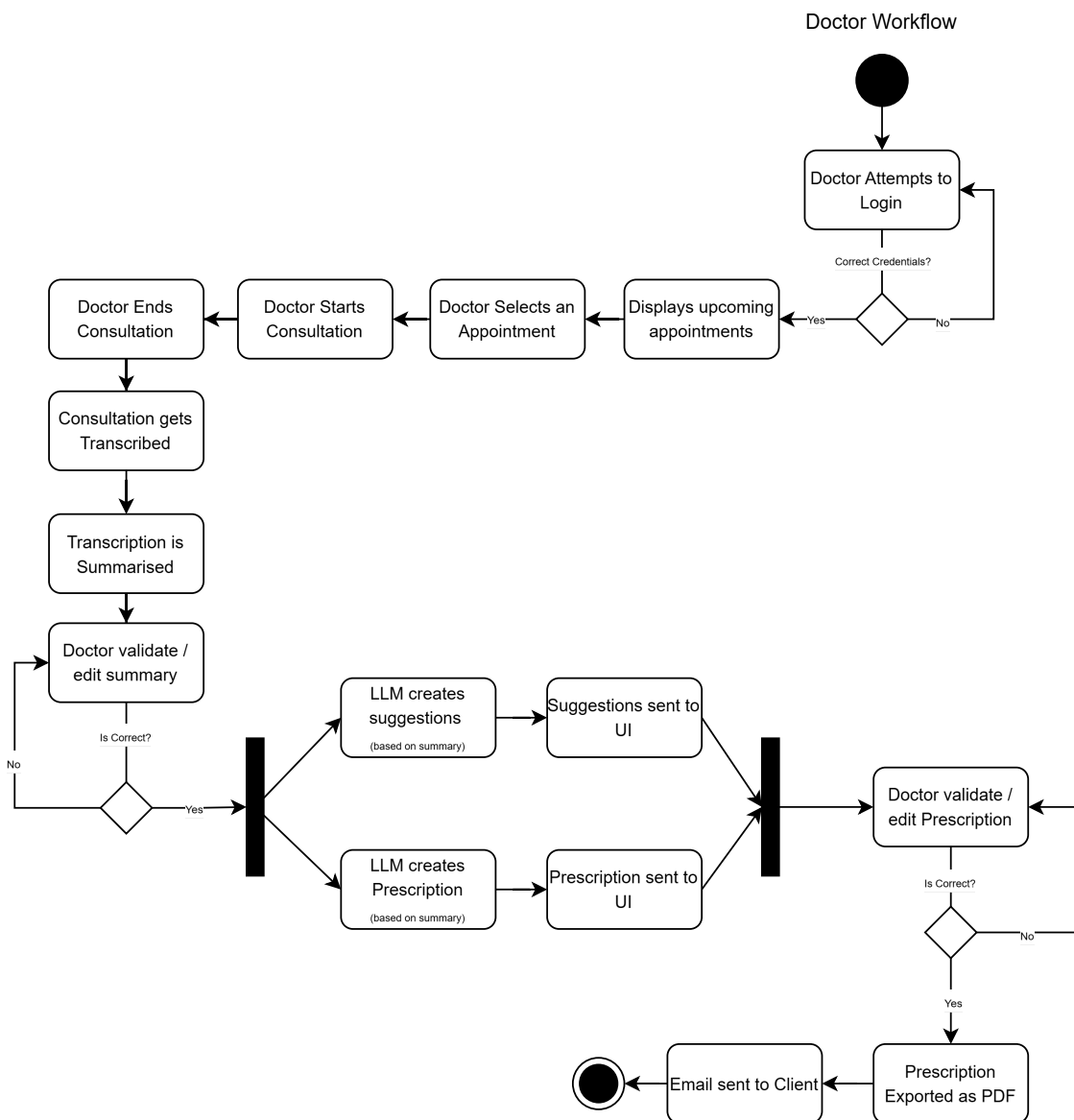


Appendix Figure C: Activity Diagram - Login

Booking - User Workflow



Appendix Figure D: Activity Diagram - Booking



Appendix Figure E: Activity Diagram - Doctor Workflow

<KEY> <Name of issue>

(eg. OPUS-0 Write a poem)

Description

Give a description of the issue and how you resolved it.

Important changes

.

Checklist

☐ My contribution follows the style guidelines of this project

☐ I have performed a self-review of my own contribution

☐ I have made corresponding changes in documentation

☐ My changes generate no new warnings or errors

☐ I have moved my task to **In Review** on Jira

Appendix Figure F: Pull Request template